Encapsulamiento

Programación Orientada a Objetos

Ing. Andrés Eduardo Paternina Ariza Esp.

Universidad del Magdalena

apaternina@unimagdalena.edu.co

¿Dónde estábamos?

En las sesiones anteriores construimos:

- **Definición de Clase y Objeto**
- **Definición de Abstracción**
- **Ejercicio inicial Abstracción**

Problema - Sistema de un cine

Un cine desea desarrollar una primera versión de su sistema. De cada película se almacena código, título, género, duración en minutos y cantidad de veces reproducida. Todas pertenecen al mismo cine, CineMag. Una película permite registrar una reproducción, consultar cuántas veces se ha reproducido y determinar si es larga, es decir, si dura más de 120 minutos.

De cada sala se almacena número, capacidad máxima, cantidad actual de personas y estado de disponibilidad. Todas las salas comparten un mismo porcentaje máximo de ocupación, inicialmente del 90 %. Una sala permite registrar el ingreso y la salida de una persona, consultar los puestos disponibles, determinar si alcanzó el límite de ocupación y evitar que ingresen más personas cuando lo alcance.

Tarjeta CRC: Película

CLASE: Película	
RESPONSABILIDADES • Guardar código, título, género, duración en minutos y veces reproducida • Registrar una reproducción • Informar cuántas veces se ha reproducido • Determinar si es larga (más de 120 minutos)	**COLABORADORES** • Ninguno por ahora: se basta sola

COMPARTIDO POR TODAS **Duración larga:** más de 120 minutos

Problema - Sistema de un cine

Un cine desea desarrollar una primera versión de su sistema. De cada película se almacena código, título, género, duración en minutos y cantidad de veces reproducida. Todas pertenecen al mismo cine, CineMag. Una película permite registrar una reproducción, consultar cuántas veces se ha reproducido y determinar si es larga, es decir, si dura más de 120 minutos.

De cada sala se almacena número, capacidad máxima, cantidad actual de personas y estado de disponibilidad. Todas las salas comparten un mismo porcentaje máximo de ocupación, inicialmente del 90 %. Una sala permite registrar el ingreso y la salida de una persona, consultar los puestos disponibles, determinar si alcanzó el límite de ocupación y evitar que ingresen más personas cuando lo alcance.

Tarjeta CRC: Sala

CLASE: Sala	
RESPONSABILIDADES • Guardar número, capacidad máxima, cantidad actual de personas y estado de disponibilidad • Registrar el ingreso de una persona • Registrar la salida de una persona • Informar los puestos disponibles • Determinar si alcanzó el límite de ocupación • Impedir el ingreso cuando alcance el límite	**COLABORADORES** • Ninguno por ahora: se basta sola

COMPARTIDO POR TODAS **Ocupación máxima:** 90 %

Los cuatro pilares de la POO

Abstracción

Quedarse con lo esencial del dominio y dejar fuera el resto.

Encapsulamiento

Herencia

Polimorfismo

La clase Película en C#

```
public class Pelicula
{
    public static int DuracionLarga = 120;
    public string Codigo { get; set; }
    public string Titulo { get; set; }
    public string Genero { get; set; }
    public int DuracionMinutos { get; set; }
    public int VecesReproducida { get; set; }
    public int Reproducciones { get { return VecesReproducida; } }
    public bool EsLarga => DuracionMinutos > DuracionLarga;
    public Pelicula(string codigo, string titulo, string genero, int duracionMinutos)
    {
        Codigo = codigo;
        Titulo = titulo;
        Genero = genero;
        DuracionMinutos = duracionMinutos;
        VecesReproducida = 0;
    }
    public void RegistrarReproduccion()
    {
        VecesReproducida++;
    }
}
```

Instanciar y modificar una Película

```
Pelicula p = new Pelicula("P001", "Inception",
                          "Ciencia ficción", 148);

Console.WriteLine(p.Titulo);  // Inception
Console.WriteLine(p.EsLarga);  // True

p.RegistrarReproduccion();
Console.WriteLine(p.Reproducciones);  // 1

p.Titulo = "Otro título";  // Se modifica arbitrariamente
p.DuracionMinutos = -50;  // Se modifica arbitrariamente
p.VecesReproducida = 1000;  // Se modifica arbitrariamente
```

¿Qué es encapsulamiento?

Esconder el **estado interno** y controlar **cómo** se modifica.

¿Qué hace?

Vuelve los atributos privados y obliga a pasar por validaciones para leerlos o cambiarlos.

¿Para qué sirve?

Garantiza que el objeto nunca esté en un estado inválido. El dominio se defiende solo.

Getters y setters

Getter

Devuelve el valor del atributo privado sin modificarlo.

Ejemplo

```
Console.WriteLine(p.Nombre);
```

Setter

Asigna un valor al atributo privado o lo transforma.

Ejemplo

```
p.Nombre = "Inception";
```

Modificadores de acceso

`public` → accesible desde fuera

`private` → solo la clase

Validación

Validación

Comprobar que un dato cumple lo esperado antes de aceptarlo en el objeto. Si no lo cumple, se rechaza.

Ejemplo

Rechazar un nombre vacío

Regla de validación

Condición concreta que un dato debe cumplir para ser válido. Cada regla se escribe como un if en el set de la property.

Ejemplo

`Nombre` no puede ser vacío

Película con Nombre encapsulado

```
public class Pelicula
{
    private string _nombre;
    public string Nombre
    {
        get { return _nombre; }
        private set
        {
            if (string.IsNullOrWhiteSpace(value))
            {
                throw new Exception("Nombre vacío");
            }
            _nombre = value;
        }
    }
    private Pelicula() { }
    public Pelicula(string nombre)
    {
        Nombre = nombre;
    }
}
```

¿Qué cambia al encapsular?

	SIN ENCAPSULAR	CON VALIDACIÓN
Atributo	Misma propiedad	`private string _nombre;`
Property	`Nombre { get; set; }` (get y set genéricos)	`Nombre` con get y set con validacion
Validación	ninguna	nombre no vacío
Nombre vacío	**posible**	**imposible**

Métodos para validar reglas en C#

Método	Qué valida	Ejemplo
`string.IsNullOrEmpty(s)`	Null o vacío. Un espacio pasa como válido.	`IsNullOrEmpty("")` → true
`string.IsNullOrWhiteSpace(s)`	Null, vacío o solo espacios.	`IsNullOrWhiteSpace(" ")` → true
`s.Length`	Largo mínimo o máximo del texto.	`"Al".Length < 3` → true
`int.TryParse(s, out n)`	Que el texto sea un número.	`TryParse("abc", out n)` → false

Reglas de validación para listas

Método	Qué valida	Ejemplo
`lista.Any()`	Tiene al menos un elemento.	`vacia.Any()` → false
`lista.Any(x => ...)`	Algún elemento cumple la regla.	`notas.Any(n => n < 3)` → true
`lista.All(x => ...)`	Todos cumplen la regla.	`notas.All(n => n >= 3)` → false
`lista.Contains(x)`	El elemento ya está en la lista.	`nombres.Contains("Ana")` → true

Any y All requieren `using System.Linq;`

Problema - Película

Un cine desea desarrollar una primera versión de su sistema. De cada película se almacena código, título, género, duración en minutos y cantidad de veces reproducida. Todas pertenecen al mismo cine, CineMag. Una película permite registrar una reproducción, consultar cuántas veces se ha reproducido y determinar si es larga, es decir, si dura más de 120 minutos.

Reglas de validación de Película

Atributo	Regla
Código	Obligatorio, no puede estar vacío y tiene mínimo 3 caracteres.
Título	Obligatorio y con mínimo 2 caracteres.
Género	Obligatorio. Solo se acepta Acción, Comedia, Drama, Terror o Animación.
Duración	Mayor que 0 y máximo 300 minutos.

Problema - Sala

De cada sala se almacena número, capacidad máxima, cantidad actual de personas y estado de disponibilidad. Todas las salas comparten un mismo porcentaje máximo de ocupación, inicialmente del 90 %. Una sala permite registrar el ingreso y la salida de una persona, consultar los puestos disponibles, determinar si alcanzó el límite de ocupación y evitar que ingresen más personas cuando lo alcance.

PROBLEMA · SALA

Reglas de validación de Sala

Atributo	Regla
Número	Debe ser mayor que 0.
Capacidad máxima	Debe ser mayor que 0.
Cantidad actual de personas	Inicia en 0 y no se modifica directamente.

Pregunta

¿Los métodos también se encapsulan?

Hasta ahora protegimos los atributos. ¿Qué pasa con las acciones de la clase?

Los métodos también llevan nivel de protección

Interfaz pequeña

Cada método lleva su modificador de acceso.

`public`: lo que otras clases llaman.

`private`: ayudas internas de la clase.

```
public class Pelicula
{
    public void RegistrarReproduccion()
    {
        ActualizarContador();
    }
    private void ActualizarContador()
    {
        VecesReproducida++;
    }
}
```

BLOQUE 2 · TEORÍA

Pregunta

¿Cómo se modifica un atributo?

Si el set es privado, otra clase no puede asignarlo directamente.

Se modifica con un método de la clase

A través de un método

El atributo queda privado.

Un método público recibe el valor nuevo y lo asigna.

Así la clase decide cuándo y cómo cambia.

Validación: si un método necesita validar un comportamiento lo hace dentro del metodo

```
public class Pelicula
{
    public string Genero { get; private set; }
    public void CambiarGenero(string genero)
    {
        Genero = genero;
    }
}
// Desde otra clase
p.CambiarGenero("Drama");
```

El método valida el comportamiento

Regla del comportamiento

Si la película ya tiene reproducciones, el género no se cambia y se avisa por pantalla.

Es una regla de la acción, no del dato, por eso va dentro del método.

```
public int VecesReproducida { get; private set; }
public string Genero { get; private set; }

public void CambiarGenero(string genero)
{
    if (VecesReproducida == 0)
    {
        Genero = genero;
    }
    else
    {
        Console.WriteLine(
            "No se puede cambiar el género " +
            "porque ya fue reproducida.");
    }
}
```

Problema - Reservas de un hotel

Un hotel desea desarrollar una primera versión de su sistema para gestionar las reservas de habitaciones.

De cada reserva se almacena:

- Código de reserva.
- Nombre del huésped.
- Tipo de habitación.
- Cantidad de huéspedes.
- Cantidad de noches.
- Precio por noche.
- Estado de la reserva.

Problema - Reglas de la reserva

La clase debe cumplir las siguientes reglas:

- El código de reserva no puede estar vacío y debe tener exactamente 6 caracteres.
- El nombre del huésped debe contener mínimo 3 caracteres.
- El tipo de habitación solo puede ser "Individual", "Doble" o "Familiar".
- La cantidad de huéspedes depende del tipo de habitación:
  - Una habitación Individual permite máximo 1 huésped.
  - Una habitación Doble permite máximo 2 huéspedes.
  - Una habitación Familiar permite máximo 5 huéspedes.
- La cantidad de huéspedes debe ser siempre mayor que 0.
- La cantidad de noches debe estar entre 1 y 30.
- El precio por noche debe ser mayor que 0.

Problema - Reglas de la reserva

La clase debe cumplir las siguientes reglas:

- El estado de la reserva solo puede ser "Pendiente", "Confirmada" o "Cancelada".
- Una reserva solo puede pasar a estado "Confirmada" si tiene al menos un huésped registrado y la cantidad de noches es válida.
- Una reserva cancelada no puede modificar la cantidad de huéspedes ni la cantidad de noches.
- Si se cambia el tipo de habitación, la nueva habitación debe soportar la cantidad de huéspedes actualmente registrados. Por ejemplo, una reserva con 4 huéspedes no puede cambiarse de Familiar a Doble.
- La cantidad de huéspedes y el tipo de habitación no deben poder modificarse libremente sin realizar estas validaciones.